

{dev/talles}

LangChain

Por: Ricardo Cuellar





¿Qué es el LangChain?

Es un framework opensource creado en 2022 por Harrison Chase. Su propósito es simplificar la construcción de aplicaciones que usan modelos de lenguaje (Ej. GPT).





Sin LangChain

Cada desarrollador manejaba el historial de mensajes, conecta con las bases de datos vectoriales, implementar tools, gestionar errores de la API y todo lo relacionado desde CERO.





Con LangChain

Se tiene estandarizado esos patrones que hacemos desde cero y se empaquetaron en componentes reutilizables.

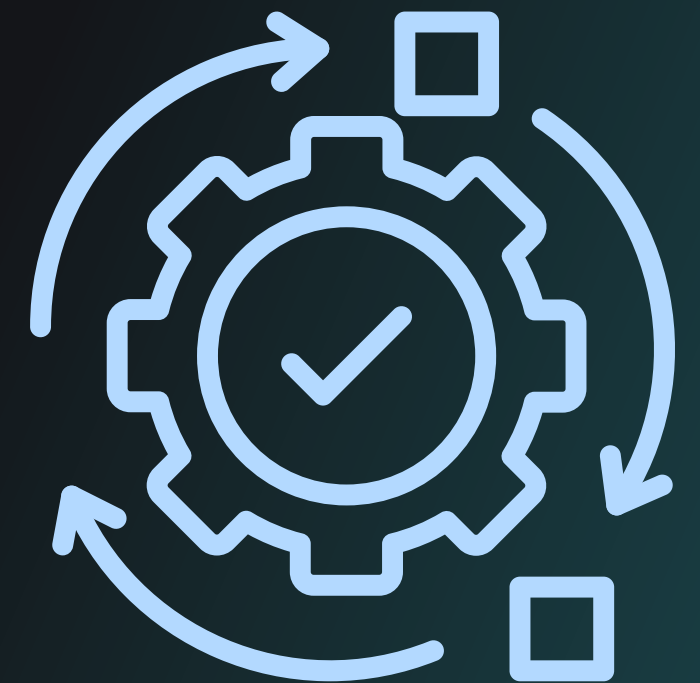
LangChain es la librería más usada en el ecosistema de IA Aplicada.



¿Para qué sirve?

Ayuda a construir aplicaciones que combinan modelos de lenguaje con otras herramientas y datos. Ejemplo:

- Chatbots avanzados
- Asistentes de consulta
- Sistemas de preguntas y respuestas sobre documentos
- Automatización de tareas con IA





¿Cómo funciona?

LangChain organiza el uso de modelos de lenguaje en componentes reutilizables:

- Models: como OpenAI o Anthropic.
- Prompts: Instrucciones que les das al modelo
- Chains: Secuencias de pasos (Ej. Preguntas → procesar → responder)
- Agents: Sistemas que deciden que hacer según la situación.
- Memory: permite recordar información en una conversación.



Ejemplo:

Cuando creamos nuestro Chat con PDFs creamos:

- División de partes.
- Embeddings para convertir texto a vectores.
- Buscar la similitud.
- Usar el modelo para generar la respuesta.

Con LangChain ahorramos todo ese tiempo, puedes integrar múltiples fuentes de datos, construir sistemas más complejos con menos código.



¿Qué es LCEL?

Son las siglas de LangChain Expression Language. Es la sintaxis de LangChain que usa el operador pipe “|” para encadenar componentes de forma declarativa.

LCEL



LCEL

Pipes en Unix

```
cat archivo.txt | grep "error" | sort
```

Pipes en LCEL

```
prompt | llm | parser
```



¿Qué es Runnable?

Todos los componentes de LangChain implementan la interfaz Runnable.
Significa que tienen los mismos métodos:
.invoke(),.batch(),.stream().

Esto es lo que hace posible encadenarlos con | .



¿Qué es Runnable?

ChatPromptTemplate → Runnable (dict → messages)

ChatOpenAI → Runnable (messages → AIMessage)

StrOutputParser → Runnable (AIMessage → string)

Encadenados: dict → [prompt] → messages → [llm] → AIMessage → [parser] → string



Gracias



www.devtalles.com



[/ricardocuellar](https://www.linkedin.com/company/ricardocuellar)



[@DevTalles](https://twitter.com/DevTalles)